TECNOLÓGICO NACIONAL DE MÉXICO

DEPARTAMENTO DE SISTEMAS Y COMPUTACIÓN

INGENIERÍA INFORMÁTICA

Karta App: Arquitectura de Persistencia

Transición del Modelo Relacional a una Base de Datos Orientada a Objetos

---

Materia: Tópicos de Bases de Datos (IFF-1026) — Unidad 2

Docente: Jonathan Sánchez Luna

Alumno: Teista García Cesar Gabriel — 22211932

Fecha: Octubre de 2026

Índice de Contenido

1. Diagnóstico del Modelo Relacional Actual	3
1.1 Análisis del esquema relacional en MySQL	3
1.2 El desfase de impedancia objeto-relacional	3
1.3 Costo de los JOINs y límites de la Primera Forma Normal	4
1.4 Identidad por valor frente a identidad por referencia	4
2. Modelo de Objetos Propuesto	5
2.1 Transformación de tablas a clases	5
2.2 Diagrama de clases UML y relaciones	5
3. Aplicación de los Pilares y Conceptos Centrales	7
3.1 Abstracción y Encapsulamiento con invariantes	7
3.2 Herencia y Polimorfismo en tiempo de ejecución	8
3.3 Identidad de Objetos (OID) y ausencia de claves foráneas	8
4. Estrategia de Persistencia	9
4.1 Modelo de persistencia por alcanzabilidad (Reachability)	9
4.2 Raíz persistente (Root) y cierre transitivo	9
4.3 Ciclo de vida: activación, ghosting y clustering físico	10
5. Definición Formal del Esquema (ODL y OQL)	11
5.1 Especificación de clases en ODL (Estándar ODMG)	11
5.2 Consultas OQL frente a su equivalente en SQL	12
6. Transacciones, Concurrencia y Evolución del Esquema	13
6.1 Escenario de concurrencia en Karta App y resolución	13
6.2 Estrategias de evolución del esquema (Eager vs Lazy)	14
7. Selección Tecnológica y Evaluación Crítica	15
7.1 Comparativa: OODBMS Puro vs ORDBMS vs RDBMS + ORM	15
7.2 Justificación de la solución adoptada	16
8. Conclusiones	17
9. Referencias Bibliográficas (APA 7)	18

1. Diagnóstico del Modelo Relacional Actual

El proyecto **Karta App** fue concebido como una plataforma digital orientada a la difusión y gestión gastronómica B2B y B2C, conectando a negocios restauranteros con clientes comensales. En su concepción inicial, la persistencia se estructuró mediante un modelo relacional normalizado sobre el motor InnoDB de MySQL, compuesto por 12 tablas (usuarios, restaurantes, sucursales, horarios, categorías, platillos, etiquetas dietéticas y analíticas).

1.1 El Desfase de Impedancia Objeto-Relacional

Al implementar la lógica de negocio en un lenguaje orientado a objetos como Python, surge de forma inmediata el fenómeno conocido en la literatura técnica como **desfase de impedancia objeto-relacional** (*Object-Relational Impedance Mismatch*). Este problema no es de índole sintáctica, sino una contradicción fundamental entre dos paradigmas incompatibles:

- **Paradigma de Programación (OO):** Organiza el dominio mediante grafos de objetos ricos en memoria, provistos de identidad por referencia, encapsulamiento de datos, relaciones de composición directa y polimorfismo dinámico.
- **Paradigma Relacional (SQL):** Organiza la información en tuplas planas pertenecientes a relaciones normalizadas, donde los datos deben ser forzosamente atómicos y la identidad se sustenta exclusivamente en claves primarias basadas en valores.

Como consecuencia, cada operación de lectura o escritura exige una costosa capa de traducción bidireccional: los objetos en memoria deben 'aplanarse' en filas para guardarse, y las filas deben 'reensamblarse' en objetos para consumirse, multiplicando la complejidad algorítmica y los puntos potenciales de inconsistencia.

1.2 El Coste de los JOINs y Límites de la Primera Forma Normal (1FN)

La Primera Forma Normal prohíbe atributos multivaluados o estructuras anidadas. Para representar el menú de un restaurante en KartaDB_RelationalModel.sql, el modelo obliga a segregar la información en tres tablas conectadas jerárquicamente: restaurants, menu_categories y dishes.

Recuperar la carta completa de un negocio con sus platillos y categorías demanda la ejecución de un árbol de operaciones JOIN:

```sql
SELECT r.name, c.name, d.name, d.price
FROM restaurants r
JOIN menu_categories c ON c.restaurant_id = r.id
JOIN dishes d ON d.category_id = c.id
WHERE r.id = 'uuid-restaurante-01';
```

Código SQL 1. Reconstrucción de un menú mediante múltiples operaciones JOIN.

A medida que la cardinalidad de platillos crece, el costo de combinar tablas intermedias satura los buffers de memoria del SGBD. Peor aún: la unidad semántica se fractura. Un restaurante no existe como entidad autónoma en la base de datos, sino como fragmentos dispersos que sólo existen reunidos durante el instante en que dura la ejecución de la consulta.

1.3 Identidad por Valor frente a Identidad por Referencia

En MySQL, la identidad de una fila depende del valor contenido en su columna clave (por ejemplo, id CHAR(36)). Dos filas con idéntico nombre y precio son la misma entidad si comparten el ID, o dos entidades distintas si sus IDs difieren. El programador se ve forzado a inventar identificadores artificiales (UUIDs o autoincrementables) y propagarlos a través de llaves foráneas para simular relaciones.

En una base de datos orientada a objetos pura, la identidad es una propiedad intrínseca e inmutable del objeto (**OID**), independiente de su estado. Dos platillos pueden tener los mismos atributos y ser distinguidos perfectamente por el motor gracias a su OID, permitiendo la navegación por punteros en tiempo constante O(1).

2. Modelo de Objetos Propuesto

Para superar las fricciones del modelo relacional, se diseñó una arquitectura orientada a objetos nativa. En lugar de tablas normalizadas conectadas por claves ajenas, el dominio se modela mediante clases persistentes interconectadas en memoria en forma de grafo acíclico dirigido.

2.1 Transformación del Esquema Relacional a Clases Persistentes

La conversión toma el núcleo del negocio gastronómico (Negocios y Menú) y sustituye la fragmentación relacional por abstracciones orientadas a objetos:

Tabla Relacional (MySQL)	Clase Persistente (Python/Durus)	Transformación Estructural
`restaurants`	`class Restaurant(Persistent)`	Encapsula estado; identidad por OID nativo.
`dishes`	`class Dish(Persistent)`	Clase base con métodos y validación de precios.
(No soportado nativamente)	`class SpecialDish(Dish)`	Subclase polimórfica que calcula descuentos al vuelo.
`menu_categories`	`PersistentList() anidada`	Tabla intermediaria absorbida por composición directa.
`FOREIGN KEY (restaurant_id)`	`Punteros / Referencias directas`	Navegación nativa entre objetos sin sentencias JOIN.

Tabla 1. Mapeo del esquema relacional MySQL al modelo de objetos persistente.

2.2 Diagrama de Clases UML y Jerarquía

El modelo de objetos se estructura bajo una jerarquía formal en la que todas las entidades del dominio heredan de la clase base Persistent suministrada por el motor Durus. A continuación se presenta la implementación de la clase principal Restaurant:

```python
class Restaurant(Persistent):
    """Clase principal que compone al Negocio y su Menú (Composición)"""
    def __init__(self, name: str, description: str, phone: str) -> None:
        super().__init__()
        self.__name: str = name
        self.__description: str = description
        self.__phone: str = phone
        self.__dishes: PersistentList = PersistentList()

    def add_dish(self, dish: Dish) -> None:
        self.__dishes.append(dish)
        self._p_note_change()

    def remove_dish(self, dish_name: str) -> None:
        """Baja con efecto en cascada simulado para la composición"""
        for dish in list(self.__dishes):
            if dish.name == dish_name:
                self.__dishes.remove(dish)
                self._p_note_change()
                break

    @property
    def name(self) -> str:
        return self.__name

    @property
    def dishes(self) -> PersistentList:
        return self.__dishes
```

Código Python 1. Implementación de la clase Restaurant con composición persistente.

En esta estructura se aprecian dos relaciones clave: una relación de **herencia simple** (SpecialDish hereda de Dish) y una relación de **composición fuerte** (un Restaurant es dueño existencial de su lista de platillos PersistentList). Si el restaurante es destruido, la colección dependiente de platillos pierde su alcanzabilidad y desaparece del almacén de datos.

3. Aplicación de los Pilares y Conceptos Centrales

La base de datos orientada a objetos en Karta App materializa los cuatro pilares de la programación orientada a objetos directamente en el almacenamiento persistente, superando las limitaciones pasivas de las tablas SQL.

3.1 Abstracción y Encapsulamiento con Invariantes de Negocio

En las bases relacionales, los datos se exponen de forma desprotegida; cualquier aplicación externa con permisos de escritura puede insertar un precio negativo en una columna price si no existe un constraint restrictivo en el motor. En nuestro modelo orientado a objetos, el estado interno se protege mediante atributos privados (prefijo de doble guión bajo __name, __price) y se expone exclusivamente mediante propiedades decoradas con @property.

El modificador setter implementa la regla de negocio que prohíbe precios negativos, asegurando que el invariante de clase nunca sea vulnerado antes de sincronizar el estado con el disco:

```
@price.setter
def price(self, new_price: float) -> None:
    if new_price < 0:
        raise ValueError("El precio no puede ser negativo.")
    self.__price = new_price
    self._p_note_change()  # Notifica al motor Durus que el objeto fue modificado
```

Código Python 2. Encapsulamiento y notificación explícita de mutación en Durus.

3.2 Herencia y Polimorfismo en Tiempo de Ejecución

El polimorfismo elimina las bifurcaciones condicionales (if / else) y las uniones complejas en SQL. La clase SpecialDish extiende a Dish incorporando un atributo privado de descuento (__discount_percentage) y sobrescribiendo dinámicamente la propiedad price:

```
class SpecialDish(Dish):
    def __init__(self, name: str, description: str, price: float, discount_percentage:
float):
        super().__init__(name, description, price)
        self.__discount_percentage = discount_percentage

    @property
    def price(self) -> float:
        # Polimorfismo: cálculo dinámico del precio con descuento al vuelo
        base_price = super().price
        return base_price * (1 - (self.__discount_percentage / 100))
```

Código Python 3. Sobreescritura polimórfica del método de cálculo de precio.

Al recorrer una colección mixta de platillos con for platillo in restaurant.dishes: print(platillo.price), el despachador dinámico de Python resuelve en tiempo de ejecución la fórmula adecuada. Para un platillo base de $120.00 con 10% de descuento, devuelve automáticamente **$108.00** de manera transparente para el consumidor.

3.3 Identidad de Objetos (OID) frente a Claves Primarias

En Durus, cada instancia que hereda de Persistent recibe un OID asignado internamente como entero de 8 bytes (int8). La identidad es independiente de los atributos: cambiar el nombre o el teléfono de un restaurante no altera su identidad ni invalida las referencias que otros objetos mantengan hacia él.

4. Estrategia de Persistencia

La persistencia es la capacidad de los datos de sobrevivir a la finalización del proceso que los creó. En los sistemas OODBMS existen tres grandes modelos de persistencia: por clase, ortogonal y por alcance. Para Karta App se seleccionó la **persistencia por alcance** (*Persistence by Reachability*).

4.1 Persistencia por Alcance y Cierre Transitivo

Bajo este modelo, el sistema gestor define una o más **raíces persistentes** (*persistent roots*). Cualquier objeto en memoria que sea alcanzable desde una raíz a través de una cadena finita de referencias se vuelve persistente de forma automática al momento de confirmar la transacción.

En nuestra implementación con Durus, la raíz se obtiene mediante connection.get_root(), la cual opera como un diccionario persistente (PersistentDict). La estructura del grafo se consolida de la siguiente manera:

1. Se enlaza la colección raíz: root['restaurants'] = PersistentList().
2. Se agregan instancias de negocio: root['restaurants'].append(restaurant).
3. Cada negocio agrega sus platillos: restaurant.add_dish(taco).
4. Al ejecutar connection.commit(), Durus calcula el cierre transitivo de alcanzabilidad y serializa en disco todo el árbol jerárquico.

4.2 Ciclo de Vida del Objeto: Activación, Desactivación y Ghosting

Durus administra la memoria física mediante la técnica de **ghosting** (objetos fantasma). Cuando la conexión abre el archivo karta_db.durus, no vuelca los megabytes o gigabytes del almacenamiento en RAM. En su lugar, crea instancias ligeras en estado GHOST que únicamente reservan el OID.

En el instante en que el código intenta leer un atributo (por ejemplo, restaurant.name), el método interceptor __getattribute__ detecta el estado fantasma y dispara la **activación** (*swapping in*), leyendo los bytes desde el archivo binario y reconstruyendo el estado del objeto en memoria. Cuando el objeto es modificado, pasa a estado UNSAVED, marcándose para sincronización en el próximo commit.

4.3 Agrupamiento Físico (Clustering) por Composición

En una base de datos de objetos, el agrupamiento físico determina si los nodos de un grafo residen en bloques de disco contiguos. En Karta App, dado que la carga de trabajo principal consiste en consultar menús completos, el almacenamiento organiza los platillos junto a su restaurante contenedor. Esto maximiza la **localidad espacial de referencia**, reduciendo las fallas de página (*page faults*) y permitiendo que una sola lectura de disco satisfaga la consulta completa del negocio.

5. Definición Formal del Esquema (ODL y OQL)

Para dotar al proyecto de rigor conforme a los estándares de la industria, el modelo se especificó bajo el estándar **ODMG 3.0** (*Object Data Management Group*), utilizando su lenguaje de definición de objetos (**ODL**) y su lenguaje de consultas declarativo (**OQL**).

5.1 Esquema en ODL (Object Definition Language)

```
module KartaApp {
    interface Dish {
        attribute string name;
        attribute string description;
        attribute float price;
        attribute string currency;
        attribute boolean is_available;
        float get_price();
    };

    interface SpecialDish : Dish {
        attribute float discount_percentage;
        float get_discounted_price();
    };

    interface Restaurant {
        attribute string name;
        attribute string description;
        attribute string phone;
        relationship list<Dish> dishes;
        void add_dish(in Dish d);
        void remove_dish(in string dish_name);
    };
};
```

Especificación ODL 1. Definición formal de las interfaces del dominio gastronómico.

5.2 Consultas OQL frente a su Equivalente en SQL

A continuación se comparan dos consultas fundamentales del sistema bajo ambos paradigmas:

Consulta 1: Navegación de rutas para obtener platillos de un restaurante

• OQL (Aprovecha expresiones de ruta sin JOIN explícito):

```
SELECT d.name, d.price FROM Restaurants r, r.dishes d WHERE r.name = 'Karta Bistro';
```

• SQL Relacional Equivalente (Requiere 2 JOINs normalizados):

```
SELECT d.name, d.price FROM restaurants r JOIN menu_categories c ON c.restaurant_id = r
.id JOIN dishes d ON d.category_id = c.id WHERE r.name = 'Karta Bistro';
```

Consulta 2: Consulta polimórfica de platillos económicos (< $110.00)

- **OQL (Polimórfica nativa; invoca el precio con descuento en SpecialDish):**

```
SELECT d.name, d.price FROM (SELECT r.dishes FROM Restaurants r) d WHERE d.price < 110.
0;
```

- **SQL Relacional Equivalente (Requiere simulación de subtipos con CASE o UNION):**

```
SELECT name, CASE WHEN discount_percentage IS NOT NULL THEN price * (1 - discount_perce
ntage / 100) ELSE price END AS final_price FROM dishes WHERE (CASE WHEN discount_percen
tage IS NOT NULL THEN price * (1 - discount_percentage / 100) ELSE price END) < 110.0;
```

6. Transacciones, Concurrencia y Evolución del Esquema

En un entorno productivo como Karta App, múltiples usuarios (clientes consultando la carta y administradores actualizando precios) interactúan concurrentemente sobre la base de datos.

6.1 Escenario de Concurrencia y Resolución

Escenario: Mientras un administrador actualiza el precio de los platillos en el restaurante 'Karta Bistro' desde el panel web, cientos de clientes consultan simultáneamente el menú desde la aplicación móvil.

Resolución en Durus: Durus implementa un modelo de aislamiento basado en **Control de Concurrencia Optimista (OCC)**. Cada cliente que abre una conexión obtiene una instantánea aislada (*snapshot*) del grafo en memoria. Los comensales leen sin bloquear el almacén. Cuando el administrador realiza connection.commit(), el motor valida si existieron conflictos de escritura. Si no hubo colisión, los cambios se asientan permanentemente; si hubiese existido concurrencia de escritura sobre el mismo objeto, se levanta una excepción ConflictError para reintentar la transacción.

6.2 Atomicidad y Reversión con abort()

El método connection.abort() garantiza que ninguna alteración errónea o incompleta corrompa el archivo binario. Si durante una actualización masiva ocurre una falla de validación, invocar abort() devuelve las instancias de PersistentList y objetos modificados al estado exacto que tenían en el último commit confirmado.

6.3 Evolución del Esquema: Conversión Inmediata frente a Diferida

Si el modelo de datos evoluciona en el futuro —por ejemplo, añadiendo un atributo allergens: List[str] a la clase Dish—, el motor debe manejar las instancias preexistentes en disco:

- **Conversión Inmediata (Eager):** Un script de migración recorre todos los platillos almacenados en karta_db.durus, les asigna una lista vacía de alérgenos y reescribe la base de datos completa. Es ideal para despliegues programados con ventana de mantenimiento.
- **Conversión Diferida (Lazy):** Se implementa el método especial de Python __setstate__(self, state) en la clase Dish. Cuando un platillo antiguo es 'despertado' del disco, el método detecta la ausencia del atributo allergens y le asigna un valor por defecto en memoria sin requerir migraciones masivas previas.

7. Selección Tecnológica y Evaluación Crítica

Para determinar la viabilidad industrial de los sistemas orientados a objetos frente a los paradigmas dominantes, se realizó una evaluación multidimensional entre tres arquitecturas representativas:

Criterio	RDBMS Puro (MySQL)	OODBMS Puro (Durus/ZODB)	Híbrido ORDBMS (PostgreSQL)
Modelo de Datos	Relaciones normalizadas planas.	Grafo de objetos nativo en memoria.	Tablas relacionales con soporte JSONB/UDT.
Desfase de Impedancia	Alto; requiere ORM externo.	Nulo; persistencia transparente directa.	Bajo; absorbe tipos estructurados.
Consultas Complejas	Sobresaliente mediante SQL optimizado.	Por navegación de punteros en grafo.	Excelente con SQL:2016 y operadores JSON.
Rendimiento en Navegación	Lento; penalizado por JOINs cruzados.	Tiempo casi constante O(1) por puntero.	Intermedio.
Ecosistema y Tooling	Gigantesco (BI, reporting, ETL, GUIs).	De nicho; herramientas de administración escasas.	Líder en la industria moderna.

Tabla 2. Comparativa arquitectónica entre paradigmas de bases de datos.

7.1 Veredicto y Justificación del Enfoque Híbrido en Producción

Para Karta App: El motor OODBMS puro (Durus) demostró ser excepcionalmente eficiente para la gestión del catálogo de restaurantes y menús anidados: simplifica el código, erradica los JOINs y permite modelar polimorfismo sin esfuerzo de configuración.

Sin embargo, para módulos que requieren analítica pesada, reportes agregados (p. ej. calcular ventas totales por mes o dashboards de métricas de usuarios), los motores relacionales siguen siendo insustituibles gracias a décadas de optimización de álgebra relacional. Por lo tanto, la recomendación de ingeniería para Karta App en escala masiva es una **arquitectura políglota/híbrida**: almacenar los grafos de menús y perfiles en un motor de objetos o documentos, y delegar la facturación y analítica a un motor relacional.

8. Conclusiones

El desarrollo de esta práctica permitió experimentar de manera empírica la transición entre el modelo relacional tradicional y el paradigma orientado a objetos en bases de datos. Los hallazgos principales se sintetizan en los siguientes puntos:

1. **Eliminación del desfase de impedancia:** Comprobar que una jerarquía completa de restaurante y platillos puede persistirse con un simple connection.commit() sin escribir mapeos ORM ni consultas SQL confirmó el inmenso valor de la persistencia por alcanzabilidad en la productividad del desarrollador.

2. **POO como ciudadano de primer orden en el almacenamiento:** Conceptos abstractos de aula como el encapsulamiento de invariantes, la identidad por OID y el polimorfismo dinámico cobraron vida real en disco a través de clases como Dish y SpecialDish.

3. **Conciencia arquitectónica:** Se comprendió por qué los OODBMS puros no dominaron el mercado masivo (falta de optimizadores SQL declarativos y tooling analítico universal), pero también cómo sus principios sobrevivieron y florecieron dentro de los motores NoSQL modernos y las extensiones objeto-relacionales actuales.

9. Referencias Bibliográficas (Formato APA 7)

- Atkinson, M., Bancilhon, F., DeWitt, D., Dittrich, K., Maier, D., & Zdonik, S. (1989). The Object-Oriented Database System Manifesto. In *Proceedings of the First International Conference on Deductive and Object-Oriented Databases (DOOD '89)* (pp. 223–240). Kyoto, Japan.

- Cattell, R. G. G. (Ed.). (2000). *The Object Data Standard: ODMG 3.0*. Morgan Kaufmann Publishers.

- Date, C. J. (2004). *An Introduction to Database Systems* (8th ed.). Pearson Addison-Wesley.

- Durus Project. (2006). *Durus: A Persistent Object System for Python*. MEMS Exchange / Sturgeon Systems. https://github.com/nascheme/durus

- Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of Database Systems* (7th ed.). Pearson Education.

- Kim, W. (Ed.). (1995). *Modern Database Systems: The Object Model, Interoperability, and Beyond*. ACM Press / Addison-Wesley.

- Ramakrishnan, R., & Gehrke, J. (2003). *Database Management Systems* (3rd ed.). McGraw-Hill.

- Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database System Concepts* (7th ed.). McGraw-Hill Education.